

PRACTICA: GESTIÓN DE HILOS

Estudiante: Johan Anibal Mamani Catacora

Código: 2024-119021

ANÁLISIS DE RESULTADOS

1. Al comparar el rendimiento de un programa multihilo con un monohilo, ¿qué diferencias observaste en el tiempo de ejecución? ¿Cómo influye la gestión adecuada de hilos en el rendimiento general del programa?

- En tareas livianas que involucran entrada/salida (como imprimir en consola), el programa monohilo suele ser más rápido o similar que el multihilo. Esto se debe al *overhead* (costo de procesamiento adicional) que genera la creación, sincronización y destrucción de los hilos. Una gestión adecuada de hilos permite aprovechar múltiples núcleos del procesador en tareas pesadas; sin embargo, si no se gestionan bien, el costo de alternar entre hilos y la espera por recursos compartidos puede reducir el rendimiento general.

2. ¿Qué funciones de la biblioteca `pthread` utilizaste en tu implementación? Explica brevemente la finalidad de al menos tres de estas funciones y su importancia en la gestión de hilos.

- Se utilizaron las siguientes funciones principales:
 - `pthread_create`: Crea e inicia la ejecución de un nuevo hilo en el sistema. Es fundamental porque sin ella no existe el paralelismo ni la concurrencia.
 - `pthread_join`: Bloquea la ejecución del hilo principal (*main*) hasta que el hilo especificado finalice. Es vital para la sincronización, evitando que el programa termine antes de que los hilos completen su trabajo.
 - `pthread_mutex_lock` / `pthread_mutex_unlock`: Bloquean y liberan un objeto *mutex* para proteger secciones críticas. Son esenciales para garantizar la exclusión mutua cuando varios hilos intentan acceder a un mismo recurso.

3. ¿Cómo se comportó el programa al aumentar el número de hilos más allá de un cierto punto? ¿Hubo un límite en el que el rendimiento no mejoró o incluso disminuyó? Explica por qué esto puede ocurrir en un entorno multihilo.

- Al aumentar el número de hilos (por ejemplo, a 10 hilos), el tiempo de ejecución no disminuyó proporcionalmente e incluso mostró un leve incremento. Existe un límite práctico debido a los cambios de contexto (*context switching*) que realiza el sistema operativo al alternar la CPU entre tantos hilos, sumado a la contención por el mutex, donde los hilos pasan más tiempo bloqueados esperando su turno para imprimir que ejecutando código útil.

4. Durante la ejecución, ¿observaste alguna condición de carrera o comportamiento inesperado? Si es así, ¿cómo impactó esto en los resultados obtenidos y qué medidas se podrían tomar para prevenirlo?

- En las primeras ejecuciones (sin el uso de *mutex*), se observó un orden impredecible en los mensajes e incluso la posibilidad de que los textos en consola se solaparan, ya que varios hilos intentaban escribir al mismo tiempo en la salida estándar. Para prevenir esta condición

de carrera (*race condition*), se implementó un mutex (`pthread_mutex_t`), envolviendo la llamada de impresión entre `pthread_mutex_lock` y `pthread_mutex_unlock` para asegurar que solo un hilo acceda a la consola a la vez.